UT1 · KOTLIN FUNDAMENTALS

Block 3

Functions

What you will learn
Declare and call functions with `fun`
Send data to a function with **parameters** and get a result back with `return`
Write short **single-expression functions** with `=`
Use **early returns** to validate data at the start of a function
Use **default parameters** and **named arguments**, and compare them with overloading
Split a program into small functions with a **single responsibility**

Until now, all our code lived inside `main`. That works for small programs, but as they grow, `main` becomes a long list of instructions that is hard to read, hard to fix and full of repeated code. A **function** is a named block of code that does one specific job. You write it once and use it as many times as you need, just by calling its name.

Functions are also the heart of Jetpack Compose: every piece of a screen (a button, a card, a whole page) is a function. Everything in this block, especially default parameters and named arguments, will appear in every Compose file you write.

Contents

Contenido

Contents....1
1. Declaring and calling functions....3
2. Input parameters....3
3. Returning a value....4
3.1 Functions that return nothing: Unit....4
3.2 return ends the function....4
4. Single-expression functions....5
5. Early returns....6
6. Default parameters and named arguments....6
6.1 Default parameters....6

6.2 Named arguments 7
7. Overloading vs default parameters 8
8. Single responsibility 8
9. Summary: Java vs Kotlin 10
10. Exercises 10
Exercise 1 · Greetings 10
Exercise 2 · Unit converter 10
Exercise 3 · Username generator 11
Exercise 4 · Guard clauses 11
Exercise 5 · Find the errors 11
Exercise 6 · Organize a product calculator using functions 11
Challenge · Refactor Block 2 12

1. Declaring and calling functions

A function is declared with the keyword `fun`, followed by its name, a pair of parentheses and a block between braces. To run it, you **call** it by writing its name followed by parentheses.

```
fun main() {
    printSeparator()
    println("Welcome to the DAM Store")
    printSeparator()
}

fun printSeparator() {
    println("======================")
}
```

Output

```
======================
Welcome to the DAM Store
======================
```

Notice that `main` is itself a function: the special one where the program starts. Also notice that `printSeparator` is declared **after** `main` and it still works: in Kotlin, the order in which functions appear in a file does not matter.

COMING FROM JAVA · Functions outside classes

In Java every method must live inside a class. In Kotlin, functions can be declared directly in the file (they are called **top-level functions**), so you do not need `public static` or a class to hold them.

TIP · Naming functions

Functions **do** something, so their names usually start with a verb, in camelCase: `printSeparator`, `calculateTotal`, `showProduct`, `isValidMark`. A good name makes a comment unnecessary.

2. Input parameters

A function becomes much more useful when it can work with different data each time. **Parameters** are the inputs of a function. Each one is declared with a name and a type, separated by a colon, the same order you already use for variables:

```kotlin
fun showProduct(productName: String, price: Double) {
    println("Product: $productName")
    println("Price: $price €")
}

fun main() {
    showProduct("Kotlin Course", 9.99)
    showProduct("Java Course", 19.99)
}
```

The values you write when you call the function (`"Kotlin Course"`, `9.99`) are called **arguments**. They are assigned to the parameters **in the same order**: the first argument goes to the first parameter, and so on. The types must match: calling `showProduct(9.99, "Kotlin Course")` does not compile.

> **WATCH OUT · Parameters are read-only**
> Inside the function, a parameter behaves like a `val`: you can read it, but not change it. `price = price * 2` does not compile. If you need a modified value, create a new variable: `val doublePrice = price * 2`.

3. Returning a value

Many functions calculate something and give the result back to whoever called them. To do this, write the **return type** after the parentheses and use `return` to send the value back:

```kotlin
fun calculateSubtotal(price: Double, quantity: Int): Double {
    val subtotal = price * quantity
    return subtotal
}

fun main() {
    val subtotal = calculateSubtotal(10.0, 3)
    println("Subtotal: $subtotal")                          // 30.0
    println("Double: ${calculateSubtotal(10.0, 3) * 2}")  // 60.0
}
```

The result of a call can be stored in a variable, used in a calculation, printed in a template or passed to another function, just like any other value.

3.1 Functions that return nothing: Unit

When a function does not return any useful value (like `printSeparator`), its return type is `Unit`, the Kotlin equivalent of Java's `void`. You do not need to write it: `fun printSeparator()` and `fun printSeparator(): Unit` are the same.

3.2 return ends the function

As soon as `return` runs, the function finishes. Any code written after it in the same block **never runs**:

```
fun calculateSubtotal(price: Double, quantity: Int): Double {
    val subtotal = price * quantity
    return subtotal
    println("Subtotal calculated")   // never runs: unreachable code
}
```

IntelliJ shows this code in grey with the warning **Unreachable code**. If you need to print something, do it **before** the `return`.

4. Single-expression functions

When the body of a function is just one expression that is returned, Kotlin lets you remove the braces and `return` and use `=` instead. The return type can also be inferred:

Java

```
fun calculateSubtotal(
    price: Double,
    quantity: Int
): Double {
    return price * quantity
}
```

Kotlin

```
fun calculateSubtotal(
    price: Double,
    quantity: Int
) = price * quantity
```

This form combines very well with `if` and `when` as expressions, which you learned in Block 2:

```
fun calculateShippingCost(subtotal: Double) = if (subtotal >= 50) 0.0 else 4.99

fun gradeFor(mark: Int): String = when (mark) {
    in 0..4 -> "Fail"
    in 5..6 -> "Pass"
    in 7..8 -> "Very good"
    in 9..10 -> "Excellent"
    else -> "Invalid mark"
}

fun main() {
    println(calculateShippingCost(42.0))   // 4.99
    println(gradeFor(8))                   // Very good
}
```

TIP · When to use it

Use the `=` form for short calculations that fit in one or two lines. If the function needs several steps or variables, use a normal block with braces and `return`: clarity is more important than saving lines.

Even in the short form, writing the return type (`: String`) is a good habit when it is not obvious.

5. Early returns

An **early return** is a `return` placed at the beginning of a function to stop it as soon as we know the answer, usually because the data is not valid or the case is special. It avoids deep nesting of `if / else` blocks.

```
fun calculateShippingCost(subtotal: Double): Double {
    if (subtotal >= 50) {
        return 0.0          // special case: we already know the answer
    }
    return 4.99
}

fun calculateDiscount(price: Double, discountPercentage: Double): Double {
    // Guard clauses: reject invalid data first
    if (price <= 0) return 0.0
    if (discountPercentage !in 0.0..100.0) return 0.0

    // From here on, we know the data is valid
    return price * discountPercentage / 100
}

fun main() {
    println(calculateShippingCost(65.0))       // 0.0
    println(calculateDiscount(80.0, 25.0))     // 20.0
    println(calculateDiscount(80.0, 150.0))    // 0.0
}
```

The checks at the start of `calculateDiscount` are called **guard clauses**: they protect the rest of the function from invalid data. Compare the same logic written with nested blocks; it works, but the important line is buried at the deepest level:

Same logic without early returns: harder to read

```
fun calculateDiscount(price: Double, discountPercentage: Double): Double {
    if (price > 0) {
        if (discountPercentage in 0.0..100.0) {
            return price * discountPercentage / 100
        } else {
            return 0.0
        }
    } else {
        return 0.0
    }
}
```

6. Default parameters and named arguments

6.1 Default parameters

A parameter can have a **default value**, written with `=` after its type. If the caller does not pass that argument, the default value is used:

```kotlin
fun showProduct(
    productName: String,
    price: Double = 9.99,
    currency: String = "€"
) {
    println("$productName: $price $currency")
}

fun main() {
    showProduct("Kotlin Course")                // Kotlin Course: 9.99 €
    showProduct("Java Course", 19.99)           // Java Course: 19.99 €
    showProduct("Python Course", 14.99, "$")    // Python Course: 14.99 $
}
```

6.2 Named arguments

When you call a function you can write the **name of the parameter** before each argument. This has two advantages: the call explains itself, and you can pass the arguments **in any order** or skip the ones that have a default value.

```kotlin
fun showProduct(
    productName: String,
    price: Double = 9.99,
    currency: String = "€"
) {
    println("$productName: $price $currency")
}

fun main() {
    showProduct(productName = "Java Course", price = 19.99)
    showProduct(price = 19.99, productName = "Java Course")   // any order
    showProduct("Swift Course", currency = "$")               // skips price
}
```

Look at the last call: we wanted to change the currency but keep the default price. Without named arguments that is impossible, because the arguments are assigned in order.

Named arguments are especially useful when a function has several parameters of the same type, where a mistake in the order would compile but give wrong results:

Java

```java
// Which number is which?
createAccount("ana", 3, 18, 5)
```

Kotlin

```kotlin
createAccount(
    username = "ana",
    level = 3,
    age = 18,
    maxDevices = 5
)
```

> **TIP · This is how Compose works**
>
> Composable functions have many parameters, almost all with default values, and they are always called with named arguments. For example: `Text(text = "Hello", fontSize = 20.sp, color = Color.Red)`. You only write the parameters you want to change; the rest keep their default values.

7. Overloading vs default parameters

Overloading means declaring several functions with the same name but different parameters (a different number of them, or different types). Kotlin chooses which one to run by looking at the arguments:

```
fun showProductId(productId: Int) {
    println("Numeric ID: $productId")
}

fun showProductId(productId: String) {
    println("Text ID: $productId")
}

fun main() {
    showProductId(1024)       // Numeric ID: 1024
    showProductId("KT-01")    // Text ID: KT-01
}
```

In Java, overloading is also the only way to make a parameter optional: you write one version with the parameter and another without it. In Kotlin, a **default parameter** does the same job with a single function:

Java

```
double calculatePrice(double price) {
    return price;
}

double calculatePrice(double price,
                      double discount) {
    return price - price * discount / 100;
}
```

Kotlin

```
fun calculatePrice(
    price: Double,
    discount: Double = 0.0
) = price - price * discount / 100
```

Use…	When…
A **default parameter**	The functions do the same job and only some data is optional
Overloading	The functions receive **different types** of data (`Int` vs `String`), or do the job in genuinely different ways

8. Single responsibility

A function should do **one thing** and do it well. This is the **Single Responsibility Principle**, the "S" of the SOLID principles of software design. Look at this function:

One function, too many jobs

```
fun processOrder(price: Double, discountPercentage: Double) {
    val discountAmount = price * discountPercentage / 100
    val finalPrice = price - discountAmount

    println("Initial price: $price")
    println("Discount: $discountAmount")
    println("Final price: $finalPrice")
    println("Order processed correctly.")
}
```

It calculates **and** prints. If tomorrow we want to show the final price on a phone screen instead of the console, we cannot reuse the calculation: it is mixed with the `println` calls. Let us split it:

```
const val PERCENTAGE_BASE = 100

fun calculateDiscountAmount(price: Double, discountPercentage: Double) =
    price * discountPercentage / PERCENTAGE_BASE

fun calculateFinalPrice(price: Double, discountAmount: Double) = price - discountAmount

fun printOrderSummary(price: Double, discountAmount: Double, finalPrice: Double) {
    println("Initial price: $price")
    println("Discount: $discountAmount")
    println("Final price: $finalPrice")
}

fun processOrder(price: Double, discountPercentage: Double) {
    val discountAmount = calculateDiscountAmount(price, discountPercentage)
    val finalPrice = calculateFinalPrice(price, discountAmount)
    printOrderSummary(price, discountAmount, finalPrice)
    println("Order processed correctly.")
}

fun main() {
    processOrder(80.0, 25.0)
}
```

Now each function has one clear job: two of them **calculate** (they receive data and return a result, without printing anything), one **displays**, and `processOrder` **coordinates** the others. The calculation functions can be reused anywhere, including in an Android app.

TIP · A quick test

If you need the word "and" to describe what a function does ("it calculates the discount **and** prints the summary"), it probably has more than one responsibility. Split it.

9. Summary: Java vs Kotlin

Concept	Java	Kotlin
Declaration	`static double total(double p, int q)`	`fun total(p: Double, q: Int): Double`
Where it lives	Always inside a class	Can be top-level (directly in the file)
No return value	`void`	`Unit` (can be omitted)
Short function	Not available	`fun total(p: Double, q: Int) = p * q`
Optional parameter	Overloaded methods	`discount: Double = 0.0`
Name the arguments	Not available	`total(p = 10.0, q = 3)`
Overloading	Yes	Yes (but defaults are usually better)

10. Exercises

Create one Kotlin file per exercise inside `UT1_KotlinBasics/src/exercises`. In every exercise, `main` should only **call** other functions: the logic goes in the functions.

Exercise 1 · Greetings

1. Write `printWelcome()`, without parameters, that prints a welcome banner.
2. Write `greet(name: String)`, which prints "Hello, <name>!".
3. Write `greetInLanguage(name: String, language: String = "en")`, which greets in English ("en"), Spanish ("es") or French ("fr"), using `when`. Call it with and without the second argument.

Exercise 2 · Unit converter

Write these functions as **single-expression functions**, with an explicit return type, and test them from `main`:

- `celsiusToFahrenheit(celsius: Double): Double` (F = C × 9 / 5 + 32)
- `kmToMiles(km: Double): Double` (1 km = 0.621371 miles; store the factor in a constant)
- `isEven(number: Int): Boolean`
- `maxOfThree(a: Int, b: Int, c: Int): Int` (hint: use `if` as an expression)

Exercise 3 · Username generator

Write `createUsername(name: String, surname: String, year: Int = 2026, separator: String = "_"): String`, which returns the username in lowercase, for example `"laura_garcia_2026"`. Hint: `name.lowercase()`.

From `main`, call it in **five different ways**: with only the required arguments, changing only the year, changing only the separator (using a named argument), with all arguments, and with all arguments named and in a different order.

Exercise 4 · Guard clauses

Write `calculateAverage(mark1: Double, mark2: Double, mark3: Double): Double`, which returns the average of three marks. Use **early returns** so that, if any mark is outside the range 0..10, the function returns `-1.0` without calculating anything. Then write `printResult(average: Double)`, which prints "Invalid marks" if it receives `-1.0`, or the average with two decimals otherwise.

Exercise 5 · Find the errors

Each fragment has a problem. Explain it and fix it.

```
// a)
fun double(number: Int) {
    return number * 2
}

// b)
fun increase(points: Int): Int {
    points = points + 10
    return points
}

// c)
fun total(price: Double, quantity: Int) = price * quantity
val t = total(3, 19.99)

// d)
fun isAdult(age: Int): Boolean {
    if (age >= 18) {
        return true
    }
}

// e)
fun showPrice(price: Double = 9.99, product: String) { println("$product: $price") }
showPrice("Kotlin Course")
```

Exercise 6 · Organize a product calculator using functions

Write a program that calculates the final price of a product with the following data:

Data	Value
Product name	"Kotlin Course"
Unit price	99.0
Quantity	2
Discount percentage	10.0
Tax percentage	7.0

The program must calculate the subtotal, the discount amount, the price after discount, the tax amount and the final price.

Requirements:

- Create functions **without parameters** to display the welcome and goodbye messages.
- Create functions **with input parameters** to display the product information and the final summary.
- Create functions **with return values** to calculate the different amounts.
- Call everything from `main`.

Challenge · Refactor Block 2

Take your solution to the **Month and season** exercise from Block 2 and reorganise it into functions:

- `monthName(month: Int): String` and `seasonOf(month: Int): String`, both as single-expression functions with `when`.
- `readMonth(): Int`, which keeps asking the user until they enter a number between 1 and 12, and returns it.
- `main`, which should have no more than four or five lines.

Then do the same with the **Number guessing game**. Which functions did you create? Could any of them be reused in the other program?